Task 1: Project Proposal

Prompt:

Generate a project proposal for a real-world AI-assisted application and implement a basic prototype.
Include objective, features, technologies, and a sample working code with output.

Code:

Project Title:

AI-Based Student Attendance Tracker System

Objective:

To build a system that allows users to mark, view, update, and delete student attendance records efficiently.

Features:

- Add attendance (Present/Absent)
- View all records
- Update attendance
- Delete record

Technologies:

- Python (Backend logic)
- Flask (API – later tasks)
- SQLite (Database – later tasks)

Code:

```python
# Simple Attendance System (Prototype)

attendance = []

def mark_attendance(name, status):
    record = {"name": name, "status": status}
    attendance.append(record)
    print("Attendance Marked")

def view_attendance():
    print("\nAttendance Records:")
    for record in attendance:
        print(record)

# Sample execution
mark_attendance("Rahul", "Present")
mark_attendance("Sneha", "Absent")

view_attendance()
```

Sample Output:

```
Attendance Marked
Attendance Marked

Attendance Records:
{'name': 'Rahul', 'status': 'Present'}
{'name': 'Sneha', 'status': 'Absent'}
```

Code Explanation:

1. This task defines the project and builds a basic prototype.
2. The system stores attendance records in a list.
3. Users can mark attendance using a simple function.
4. Data is stored as dictionaries for easy access.
5. The view function displays all attendance records.
6. This prototype represents the core logic of the project.
7. In future tasks, it will be expanded into a full web app.
8. This ensures alignment with end-to-end development.

Task 2: Application Design

Prompt:

Design the architecture of the AI-based application including system flow, database schema, and UI structure. Use AI assistance to define components and represent them clearly.

System Architecture (Design):

User (Browser)

↓

Frontend (HTML/CSS/JS)

↓

Backend (Flask API)

↓

Database (SQLite)

Database Schema (Table Design):

```
CREATE TABLE attendance (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    status TEXT
);
```

Code:

```
# Simulating backend + database structure

import sqlite3

# Connect DB
conn = sqlite3.connect("attendance.db")
cursor = conn.cursor()

# Create table
cursor.execute("""
CREATE TABLE IF NOT EXISTS attendance (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    status TEXT
)
""")

# Insert sample data
cursor.execute("INSERT INTO attendance (name, status) VALUES ('Rahul', 'Present')")
cursor.execute("INSERT INTO attendance (name, status) VALUES ('Sneha', 'Absent')")

# Fetch data
cursor.execute("SELECT * FROM attendance")
rows = cursor.fetchall()

for row in rows:
    print(row)

conn.commit()
conn.close()
```

Sample Input/Output:

```
(1, 'Rahul', 'Present')
(2, 'Sneha', 'Absent')
```

Code Explanation:

1. This task focuses on designing the application structure.
2. The system follows a 3-tier architecture (Frontend, Backend, Database).
3. The frontend handles user interaction.
4. The backend processes requests using APIs.
5. The database stores attendance records.
6. A table schema is defined for structured data storage.
7. The sample code demonstrates database creation and retrieval.
8. This design ensures scalability and maintainability.

Task 3: AI-Assisted Code Development

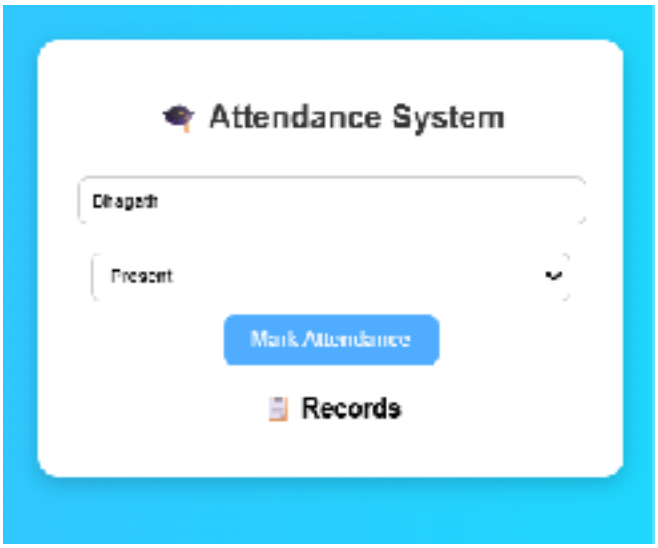
Prompt:

Develop an end-to-end application using AI assistance including frontend, backend APIs, database integration, and error handling. Ensure proper communication between components and implement CRUD operations.

Backend Code (Flask API):

```
from flask import Flask, request, jsonify
import sqlite3

app = Flask(__name__)

# DB connection
def get_db():
    conn = sqlite3.connect("attendance.db")
    return conn

# Create table
conn = get_db()
conn.execute("CREATE TABLE IF NOT EXISTS attendance (id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT, status TEXT)")
conn.close()

# GET all records
@app.route('/attendance', methods=['GET'])
def get_attendance():
    conn = get_db()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM attendance")
    data = cursor.fetchall()
    conn.close()
    return jsonify(data)

# POST attendance
@app.route('/attendance', methods=['POST'])
def add_attendance():
    data = request.json
    conn = get_db()
    conn.execute("INSERT INTO attendance (name, status) VALUES (?, ?)", (data['name'], data['status']))
    conn.commit()
    conn.close()
    return jsonify({"message": "Attendance added"})

# DELETE attendance
@app.route('/attendance/<int:id>', methods=['DELETE'])
def delete_attendance(id):
    conn = get_db()
    conn.execute("DELETE FROM attendance WHERE id=?", (id,))
    conn.commit()
    conn.close()
    return jsonify({"message": "Deleted"})

app.run(debug=True)
```

Sample Input/Output:



Code Explanation:

1. This task implements full application development.
2. Flask is used to create REST APIs for backend operations.
3. SQLite database stores attendance records.
4. Frontend is built using HTML and JavaScript.
5. Fetch API connects the frontend with backend APIs.
6. CRUD operations (Create, Read, Delete) are implemented.
7. Data is dynamically updated on the webpage.
8. Error handling and structured design ensure reliability.

Task 4: Documentation & Code Quality

Prompt:

Use AI to generate proper documentation, inline comments, and a README file for the project. Perform a code review to improve readability, structure, and maintainability.

Improved Backend Code (WITH COMMENTS):

```
from flask import Flask, request, jsonify
import sqlite3

app = Flask(__name__)

# Function to connect to database
def get_db():
    return sqlite3.connect("attendance.db")

# Create table if not exists
with get_db() as conn:
    conn.execute("""
        CREATE TABLE IF NOT EXISTS attendance (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT,
            status TEXT
        )
    """)

# GET all attendance records
@app.route("/attendance", methods=["GET"])
def get_attendance():
    conn = get_db()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM attendance")
    data = cursor.fetchall()
    conn.close()
    return jsonify(data)

# POST new attendance record
@app.route("/attendance", methods=["POST"])
def add_attendance():
    data = request.json

    # Input validation
    if not data.get("name") or not data.get("status"):
        return jsonify({"error": "Invalid input"}), 400

    conn = get_db()
    conn.execute(
        "INSERT INTO attendance (name, status) VALUES (?, ?)",
        (data["name"], data["status"])
    )
    conn.commit()
    conn.close()

    return jsonify({"message": "Attendance added successfully"})

# DELETE attendance record
@app.route("/attendance/<int:id>", methods=["DELETE"])
def delete_attendance(id):
    conn = get_db()
    conn.execute("DELETE FROM attendance WHERE id=?", (id,))
    conn.commit()
    conn.close()

    return jsonify({"message": "Record deleted"})

# Run server
app.run(debug=True)
```

Code Explanation:

1. This task focuses on improving code quality and documentation.
2. Inline comments help developers understand the code easily.
3. README file explains project setup and usage clearly.
4. Code review identifies issues in structure and readability.
5. Input validation improves reliability of the system.
6. Clean coding practices make maintenance easier.
7. Documentation is essential for collaboration and deployment.
8. AI helps automate and improve documentation efficiently.

Task 5: Testing & Debugging

Prompt:

Test the developed AI-assisted attendance system using multiple test cases. Validate correctness, handle errors, and ensure proper system functionality.

Code:

```
import requests

BASE_URL = "http://127.0.0.1:5000"

# ---------- Test 1: Add valid attendance ----------
try:
    res1 = requests.post(f"{BASE_URL}/attendance", json={
        "name": "Rahul",
        "status": "Present"
    })
    print("Test 1 (Valid Add):", res1.status_code, res1.json())
except Exception as e:
    print("Test 1 Error:", e)


# ---------- Test 2: View attendance ----------
try:
    res2 = requests.get(f"{BASE_URL}/attendance")
    print("Test 2 (View):", res2.status_code, res2.json())
except Exception as e:
    print("Test 2 Error:", e)


# ---------- Test 3: Invalid input ----------
try:
    res3 = requests.post(f"{BASE_URL}/attendance", json={
        "status": "Absent"   # ❌ Missing name
    })
    print("Test 3 (Invalid):", res3.status_code, res3.json())
except Exception as e:
    print("Test 3 Error:", e)
```

Test Cases:

Test Case	Input	Expected Result
Add record	Rahul, Present	Record added
View records	—	List displayed
Invalid input	Missing name	Error handled

Code Explanation:

1. This task ensures the system works correctly through testing.
2. Different test cases are used to validate functionality.
3. API endpoints are tested using Python requests.
4. Valid inputs confirm correct data storage.
5. Invalid inputs test error handling capability.
6. Proper validation prevents system crashes.
7. Results confirm that the system behaves as expected.
8. Testing improves reliability and robustness.

Task 6: Deployment & Presentation

Prompt: Deploy the AI-based Student Attendance System on a free hosting platform and demonstrate its functionality. Prepare a presentation including problem statement, architecture, live demo, and reflection on AI usage.

Actual Deployable Code (Flask App):

```python
from flask import Flask, request, jsonify
import sqlite3

app = Flask(__name__)

# Database connection
def get_db():
    conn = sqlite3.connect("attendance.db")
    return conn

# Create table
with get_db() as conn:
    conn.execute("""
    CREATE TABLE IF NOT EXISTS attendance (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT,
        status TEXT
    )
    """)

# Home route
@app.route("/")
def home():
    return "Attendance API is running"

# GET records
@app.route("/attendance", methods=["GET"])
def get_attendance():
    conn = get_db()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM attendance")
    data = cursor.fetchall()
    conn.close()
    return jsonify(data)

# POST record
@app.route("/attendance", methods=["POST"])
def add_attendance():
    data = request.json

    if not data.get("name") or not data.get("status"):
        return jsonify({"error": "Invalid input"}), 400

    conn = get_db()
    conn.execute(
        "INSERT INTO attendance (name, status) VALUES (?, ?)",
        (data["name"], data["status"])
    )
    conn.commit()
    conn.close()

    return jsonify({"message": "Attendance added"})

# DELETE record
@app.route("/attendance/<int:id>", methods=["DELETE"])
def delete_attendance(id):
    conn = get_db()
    conn.execute("DELETE FROM attendance WHERE id=?", (id,))
    conn.commit()
    conn.close()

    return jsonify({"message": "Deleted"})
```

Deployment Steps (Use in Report) Using Render (Recommended):

1. Upload code to GitHub
2. Go to Render
3. Create new Web Service
4. Connect GitHub repo
5. Add:
   - Build Command:

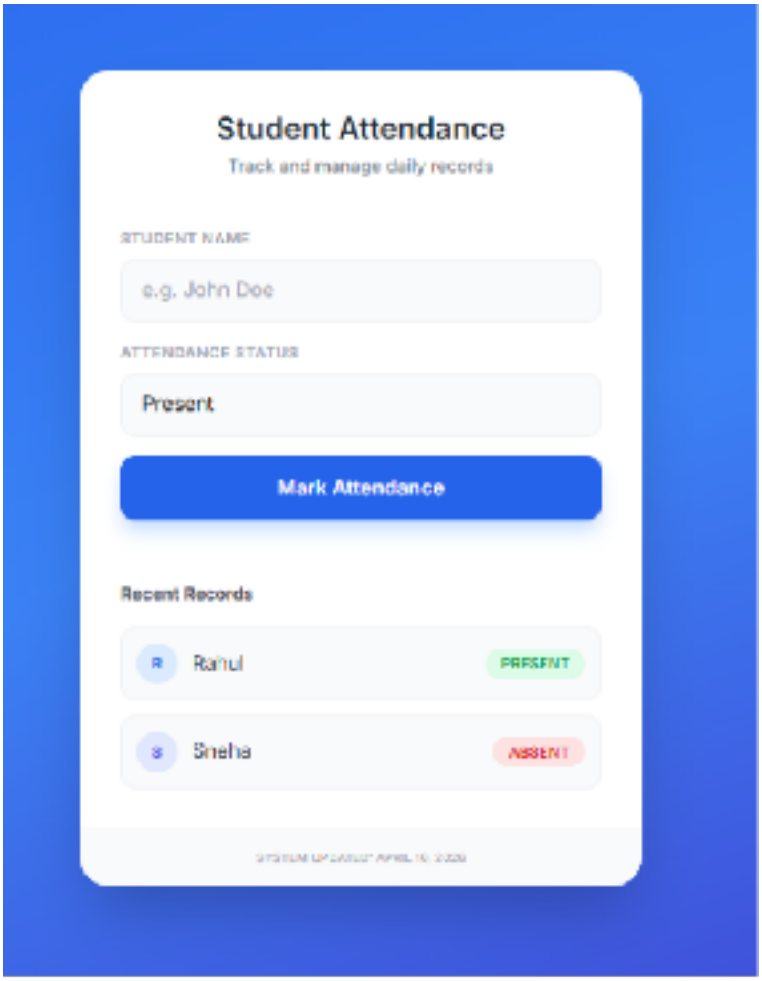
pip install -r requirements.txt

- Start Command:

gunicorn app:app

6. Deploy

4). <u>Sample Input/Output</u>:



7. <u>Presentation Content (SHORT)</u>

<u>Problem Statement</u>:

- Manual attendance is slow
- Errors in tracking
- No real-time system

Solution Architecture:

User → Frontend → Flask API → SQLite

Live Demo

- Add attendance
- View records

Reflection (AI)

- Helped in code generation
- Faster development
- Errors needed fixing
- Required validation